Lecture 3

Software Architecture

Institute of Computer Science & Information Technology,
Faculty of Management & Computer Sciences,
The University of Agriculture, Peshawar, Pakistan.

Software Architecture

- The architecture of a software system is a metaphor, analogous to the architecture of a building.
- Software architecture refers to the fundamental structures of a software system and the discipline of creating such structures and systems
- The architecture of a software system consists of it structures, the decomposition into components, and their interfaces and relationships.

The burden is on software

- Hardware and compilers can keep up the pace needed.
- From now on…
  - In shared memory programs:
    - Start a single process and fork threads.
    - Threads carry out tasks.
  - In distributed memory programs:
    - Start multiple processes.
    - Processes carry out tasks.

Parallel Programming Models

1. Single Program Multiple Data (SPMD)

- A **single executable program** runs on multiple processors.
- Each processor executes the **same program**, but may follow **different branches** depending on data or processor ID.

2. Multiple Program Multiple Data (MPMD)

- Different processors execute **different programs**.
- Each program may operate on **different datasets**.

- Most Modern MPI based parallel programs follows SPMD model using MIMD architectures.

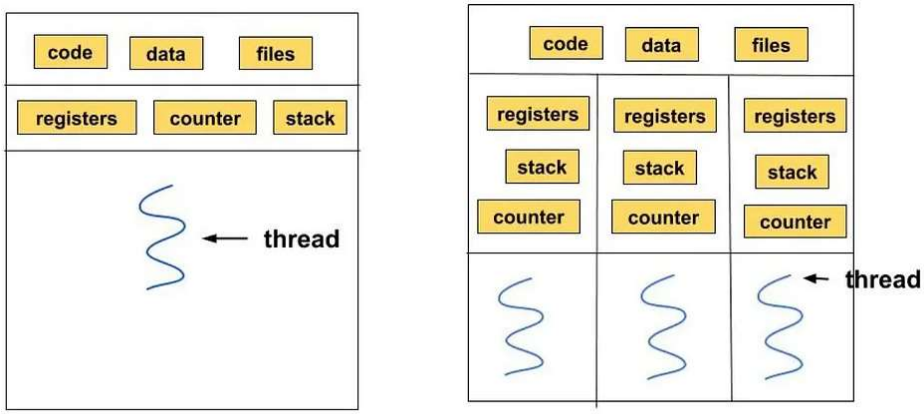
Writing Parallel Programs

1. Divide the work among the processes/threads
   (a) so each process/thread gets roughly the same amount of work (load balancing)
   (b) and communication is minimized.
2. Arrange for the processes/threads to synchronize.
3. Arrange for communication among processes/threads.

```
double x[n], y[n];
...
for (i = 0; i < n; i++)
    x[i] += y[i];
```

Process and Thread

- Process: A program in execution is called process.
- Thread: is the segment of a process, mean a process can have multiple threads and these multiple threads are contained within a process.



Single-threaded process

Multi-threaded process

Example

- In a word processor,
  - a thread may check spelling and grammar while
  - another thread processes user input (keystrokes), while
  - another third thread loads images from the hard drive, and a
  - fourth does periodic automatic backup of the file being edited.

Process Vs Thread

Process	Thread
A process is the instance of a program executed by one or many threads.	A thread is a basic unit of CPU utilisation, which consists of its own thread ID, a program counter, a register and a stack.
A process may contain multiple threads depending on the operating system.	A thread is the smallest unit of execution within a process.
In a multiprocessing environment, multiple processes do not share resources such as memory with each other.	Multiple threads of a given process running concurrently can share resources such as memory with each other.
Processes take more time for context switching.	Threads take less time for context switching.
Processes take more time for creation and termination.	Threads take lesser time for creation and termination.
Processes consume more resources.	Threads consume fewer resources.
Process communication is complex.	Thread communication is much easier and efficient.

Multithreading

- In computer architecture, multithreading is the ability of single CPU to provide multiple threads of execution concurrently, supported by the operating system.
- Multithreading aims to increase utilization of a single core by using thread-level parallelism.
- Multithreading allow for multiple request to be satisfied simultaneously, without having to service requests sequentially.

Shared Memory

- Dynamic threads
  - Master thread waits for work, forks new threads, and when threads are done, they terminate.
  - Efficient use of resources, but thread creation and termination is time consuming.
- Static threads
  - Pool of threads created and are allocated work, but do not terminate until cleanup.
  - Better performance, but potential waste of system resources.

Process and Message Passing

- Numerous programming languages (message passing paradigm) and libraries have been developed for explicit parallel programming.
- The message passing programming paradigm is one of the oldest and most widely used approaches for programming parallel computers.
- There are two key attributes that characterize the message-passing programming paradigm.
  - First, it assumes a partitioned address space (each process typically has its own private address space)
  - Second, it supports explicit communication (Processes must explicitly send messages to communicate with each other, and they must explicitly handle incoming messages).

Partitioned Address Space

- The logical view of a machine supporting the message-passing paradigm consists of p processes, each with its own address space.

- Instances of such a view come naturally from clustered workstations and non-shared address space multi-computers.

Building Block: Send & Receive Operations

- The **interactions** are accomplished by sending and receiving messages,
- The prototype of these operations are defined as follows
- *send(void*sendbuf, int nelems, int dest)*
- *receive(void*recvbuf, int nelems, int source)*
- The sendbuf points to a buffer that stores the data to be sent.
- The recvbuf points to buffer that stores the data to be received.
- The dest is the identifier of the process that receives the data.
- The source is the identifier of the process that send the data

Nondeterminism

P0	P1
a = 100;	Receive(&a,1,0)
send(&a, 1,1);	Printf("%d\n",a);
a=0;	

• The important thing to note is that process P0 changes the value of a to 0 immediately following the send.

•The semantics of the send operation require that the value received by process P1 must be 100 as opposed to 0.

Structure of Message-Passing Programs

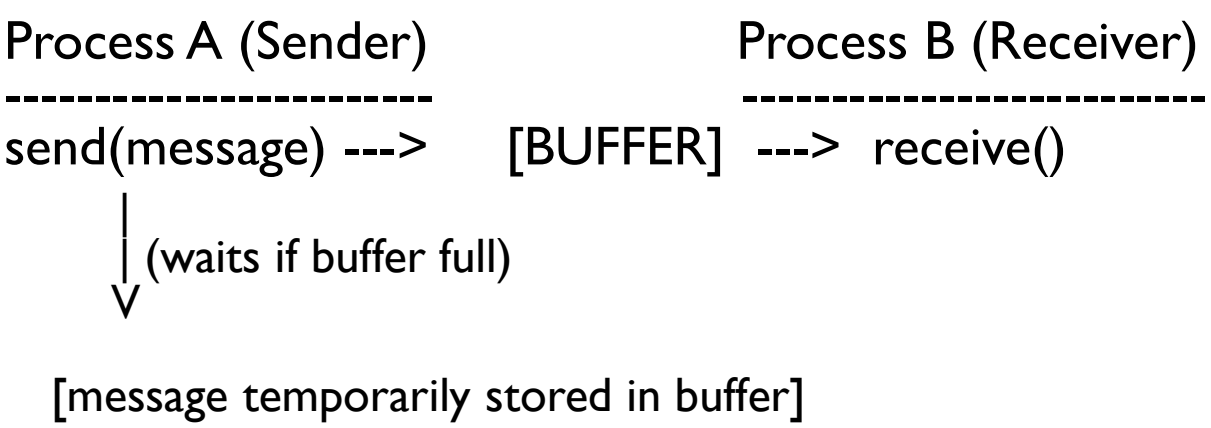
- Message-passing programs are often written using the:
  - **Asynchronous** paradigm: all the concurrent tasks execute asynchronously (no coordination) and
    - make it possibly to implement any parallel algorithm.
    - However, such programs can be harder to understand, and can have non deterministic behavior due to race conditions.
  - **Loosely synchronous:** In such programs, tasks or subsets of tasks synchronized to perform interactions.
    - However, between these interactions, tasks execute completely asynchronously.
    - Loosely synchronous programs are a good compromise between Asynchronous and Synchronous.

Message Passing Types

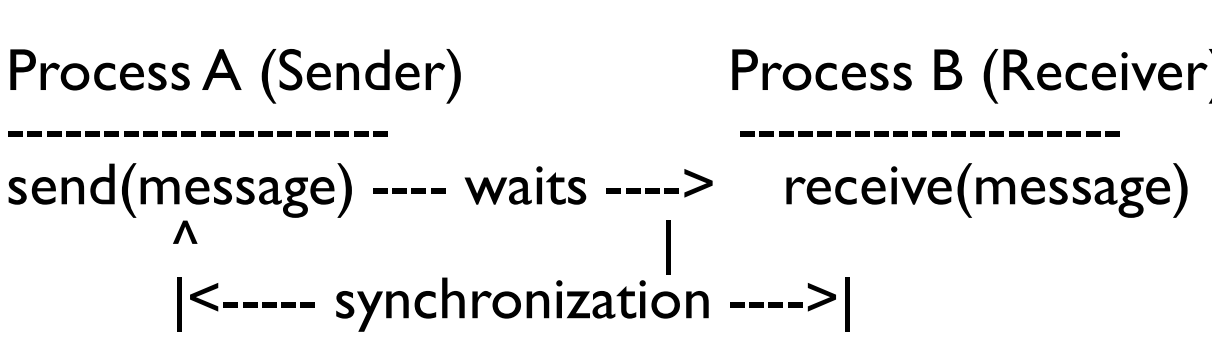
- Message passing is a method for processes to communicate by sending and receiving messages. It is how two processes (say Process A and Process B) exchange data via send and receive operations. It can be classified based on two factors i.e. blocking vs non-blocking, and buffered vs non-buffered.
- 1. Blocking vs Non-blocking
  - Blocking: The process waits (is paused) until the operation completes.
  - Non-blocking: The process continues execution immediately after issuing the operation.
- 2. Buffered vs Non-buffered
  - Buffered: Messages are stored temporarily in a queue or buffer until the receiver picks them up.
  - Non-buffered (Direct): The message must be delivered immediately — sender and receiver must synchronize.

Blocking Buffered Message Passing

- Description:
  - Sender waits only if the buffer is full.
  - Receiver waits if the buffer is empty.
- If the buffer has space, the sender sends the message and continues.
  - The buffer acts as a temporary mailbox, so sender and receiver do not need to synchronize for every send/recieve.
-



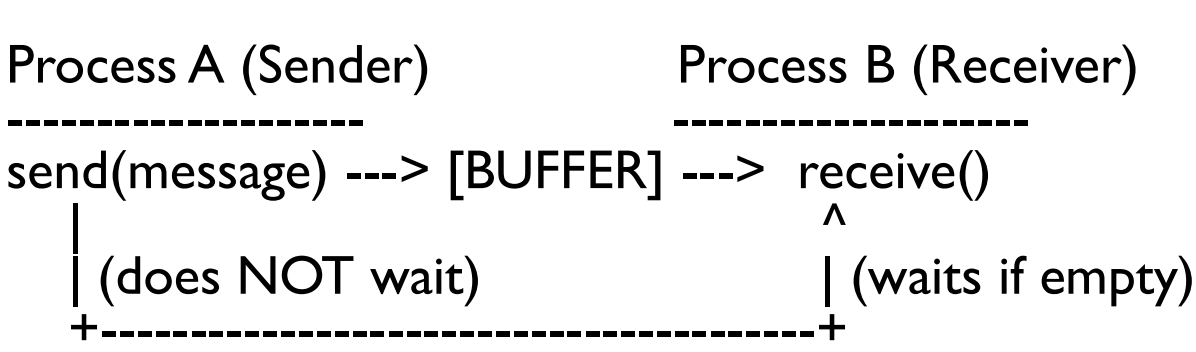
Blocking Non-Buffered Message Passing

- Description:
  - No buffer exists.
  - Sender and receiver must both be ready.
  - The sender waits until the receiver has called receive(); this is called rendezvous communication.
- 

- Both processes **block** until the message is transferred.

Non-Blocking Buffered Message Passing

- Description:
  - Sender places the message in a buffer and continues immediately.
  - Receiver reads the message later.
  - Buffer must have enough space, otherwise message loss or error can occur.



- The sender never blocks; receiver may block only if buffer is empty.

Non-Blocking Non-Buffered Message Passing

- Description:
  - No buffer exists.
  - Sender sends and continues even if receiver isn't ready.
  - If receiver isn't ready, message is lost or an error occurs.
-

Process A (Sender)	Process B (Receiver)
send(message) ---> X (no buffer)	receive()

(Sender does NOT wait;
message may be lost if receiver isn't ready)

Summary Table

Type	Sender Blocks?	Receiver Blocks?	Buffer Exists?	Synchronization Needed?
Blocking Buffered	Only if full	If empty	Yes	No
Blocking Non-Buffered	Yes (until recv ready)	Yes (until send ready)	No	Yes
Non-Blocking Buffered	No	If empty	Yes	No
Non-Blocking Non-Buffered	No	No (but may miss msg)	No	No (unreliable)

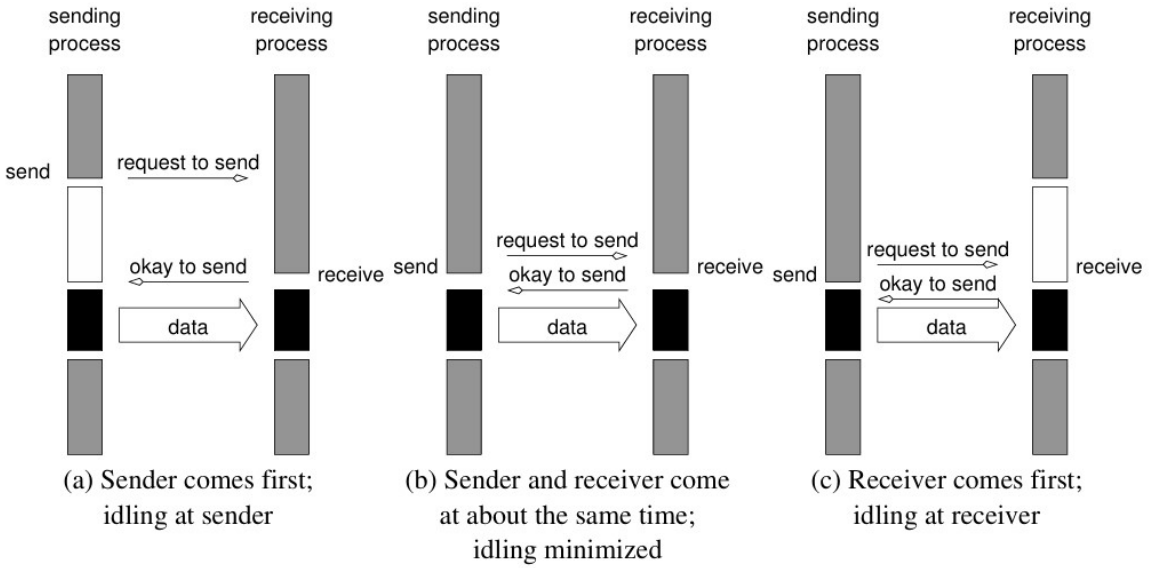
Blocking Message Passing Operations

- A simple solution to the Nondeterministic behavior presented in the code fragment above is for the send operation to return only when it is semantically safe to do so.
- It simply means that the sending operation blocks until it can guarantee that the semantics will not be violated on return irrespective of what happens in the program subsequently. There are two mechanism by which this can be achieved.
  1. Blocking non-buffered send/receive
  2. Blocking buffered send/receive

Blocking non-buffered Send/Receive

- In the first case, the send operation does not return until the matching receive has been encountered at the receiving process.
- When this happens, the message is sent and the send operation returns upon completion of the communication operation.
- Typically, this process involves a handshake between the sending and receiving processes.
- The sending process sends a request to communicate to the receiving process.
- Since, there are no buffers used at either sending or receiving ends, this is also referred to as a non-buffered blocking operations.

Blocking non-buffered message passing



Handshake for a blocking non-buffered send/receive operation. It is easy to see that in cases where sender and receiver do not reach communication point at similar times, there can be considerable idling overheads.

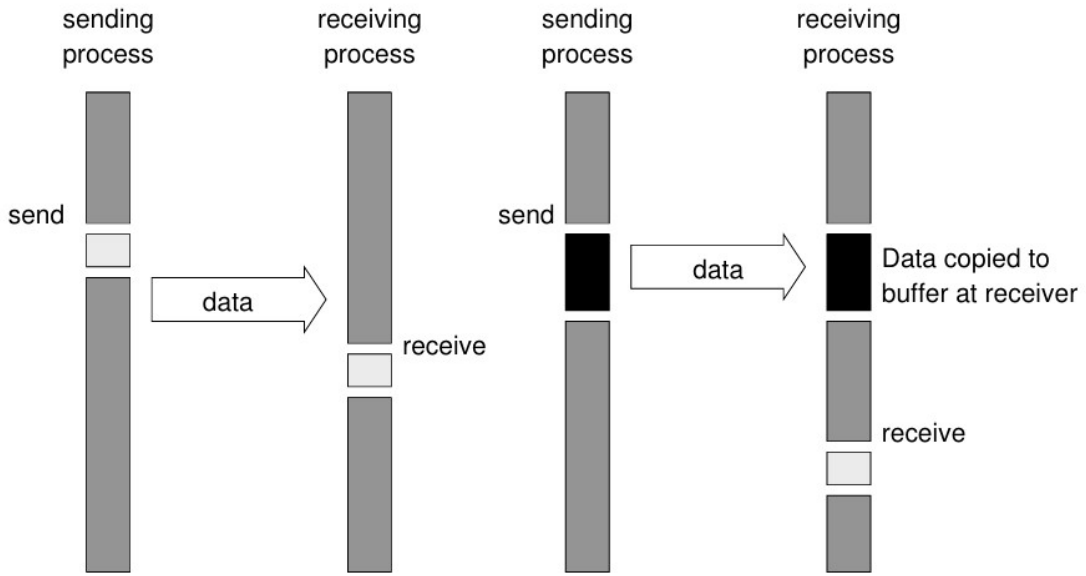
Blocking non-buffered Send/Receive (1)

- In cases (a) and (c), we notice that there is considerable idling at the sending and receiving process.
- It is also clear from the figures that a blocking non-buffered protocol is suitable when the send and receive are posted at roughly the same time.
- However, in an asynchronous environment, this may be impossible to predict . This idling overhead is one of the major drawbacks of this protocol.

Blocking Buffered Send/Receive

- A simple solution to the idling and deadlocking problem outlined above is to rely on buffers at the sending and receiving ends.
- We start with a simple case in which the sender has a buffer pre-allocated for communication messages.
- On encountering a send operations, the sender simply copies the data into the designated buffer and returns after the copy operation has been completed.

Buffered Blocking Message Passing Operation



Blocking buffered transfer protocols: (a) in the presence of communication hardware with buffers at send and receive ends; and (b) in the absence of communication hardware, sender interrupts receiver and deposits data in buffer at receiver end.

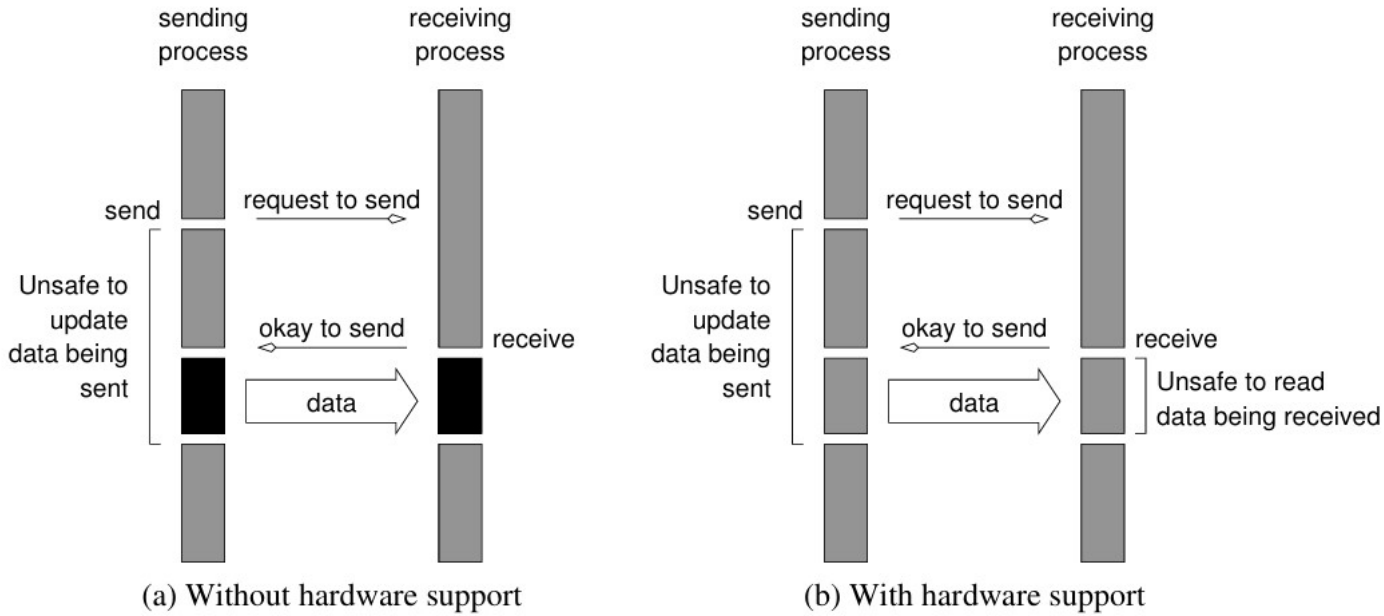
Blocking Buffered Send/Receive (1)

- The sender process can now continue with the program knowing that any changes to the data will not impact program semantics.
- Note that at the receiving end, the data cannot be stored directly at the target location since this would violate program semantics.
- Instead, the data is copied into a buffer at the receiver as well.

Non-Blocking Message Passing Operations

- The programmer must ensure semantics of the send and receive.
- This class of non-blocking protocols returns from the send or receive operation before it is semantically safe to do so.
- Message passing libraries typically provide both blocking and non-blocking primitives

Non-Blocking Message Passing Operations



Non-blocking non-buffered send and receive operations (a) in absence of communication hardware; (b) in presence of communication hardware.

Send and Receive Protocols

